MARLOM CAMARAN LUCAS

VERIFICADOR DE PREÇOS: DESENVOLVIMENTO E VALIDAÇÃO DE UM APLICATIVO ANDROID INTEGRADO A UMA API REST

Alvorada

Novembro/2026

Resumo

A consulta de preços de produtos no comércio depende da disponibilidade de informações claras e atualizadas para consumidores e funcionários. Este trabalho apresenta o desenvolvimento e a validação de um sistema de verificação de preços por código de barras, composto por um aplicativo Android e um backend conectado a banco de dados relacional. O objetivo geral consistiu em construir um protótipo capaz de identificar códigos de barras pela câmera do smartphone, consultar dados pela rede Wi-Fi e exibir nome, descrição, preço e imagem do produto. Adotou-se um processo incremental organizado por Kanban, com tarefas acompanhadas no Trello e ciclos sucessivos de implementação e testes. O aplicativo foi desenvolvido em Kotlin, Jetpack Compose e arquitetura MVVM, com CameraX e ML Kit para leitura, Retrofit e Moshi para comunicação e Coil para imagens. O servidor utiliza Python, Django, Django REST Framework e MySQL, com administração de produtos pelo Django Admin. A validação reuniu 115 testes automatizados do backend, 25 testes unitários Android e 13 cenários integrados executados em um Motorola Moto G22 com Android 12. Todos os testes informados foram aprovados, abrangendo consultas, imagens, atualização de preços, alterações cadastrais e tratamento de erros. Os resultados demonstram a viabilidade técnica do protótipo em rede local e fornecem uma base para futuras avaliações em outros ambientes de implantação.

Palavras-chave: engenharia de software; código de barras; aplicativo android.

1. INTRODUÇÃO

A disponibilização de preços corretos e acessíveis é uma necessidade recorrente no comércio varejista, especialmente em supermercados, nos quais a variedade de mercadorias pode dificultar a identificação imediata dos valores. A Lei nº 10.962/2004 estabelece modalidades de afixação de preços de produtos e serviços para o consumidor, reforçando a relevância da informação de preço no ponto de venda (BRASIL, 2004).

A ausência de etiquetas, a divergência entre o preço exposto e o cadastrado ou a indisponibilidade de equipamentos de consulta podem prejudicar a experiência de compra. Nesse contexto, a utilização da câmera de smartphones para leitura de códigos de barras permite estudar uma alternativa móvel de consulta, desde que haja acesso a uma base de produtos atualizada e comunicação confiável com o servidor.

O objetivo geral deste trabalho foi desenvolver e validar um protótipo de software que permitisse consultar preços de produtos a partir de códigos de barras, por meio de um aplicativo Android integrado a uma API REST e a um banco de dados MySQL, em ambiente de rede local.

Como objetivos específicos, buscou-se: especificar o escopo e as funcionalidades; estruturar um backend com cadastro administrativo de produtos; implementar a leitura automática e a consulta manual no Android; integrar os componentes por HTTP e JSON; incorporar a exibição opcional de imagens; e verificar o funcionamento por testes automatizados e integrados.

O projeto justifica-se pela possibilidade de tornar mais acessível a consulta às informações de mercadorias e pela oportunidade de aplicar conceitos de Engenharia de Software em um produto funcional. A construção inclui separação de responsabilidades, documentação de API, controle de estados, tratamento de erros, testes de regressão e avaliação do fluxo completo no dispositivo físico.

2. ESPECIFICAÇÕES INICIAIS DO SOFTWARE

2.1 Escopo do produto

O Verificador de Preços foi concebido como um sistema cliente-servidor para consulta de informações comerciais cadastradas. O usuário utiliza o aplicativo Android para ler o código de barras com a câmera ou informá-lo manualmente. O aplicativo solicita os dados ao backend, que consulta o registro correspondente no MySQL e retorna uma resposta estruturada em JSON.

A versão validada tem como foco a consulta individual por código de barras convencional, sem leitura de QR Code. O aplicativo não realiza vendas, pagamento, controle de estoque, emissão de documentos fiscais ou alteração de preços. O gerenciamento cadastral é reservado a um administrador autenticado no Django Admin, que pode cadastrar, editar e excluir produtos e associar imagens opcionais.

Para limitar a complexidade e permitir a comprovação do funcionamento, a validação foi concentrada em dispositivos conectados à mesma rede Wi-Fi do servidor. O resultado é um protótipo acadêmico funcional, não uma solução homologada para implantação comercial, acesso público pela internet ou todas as versões do Android.

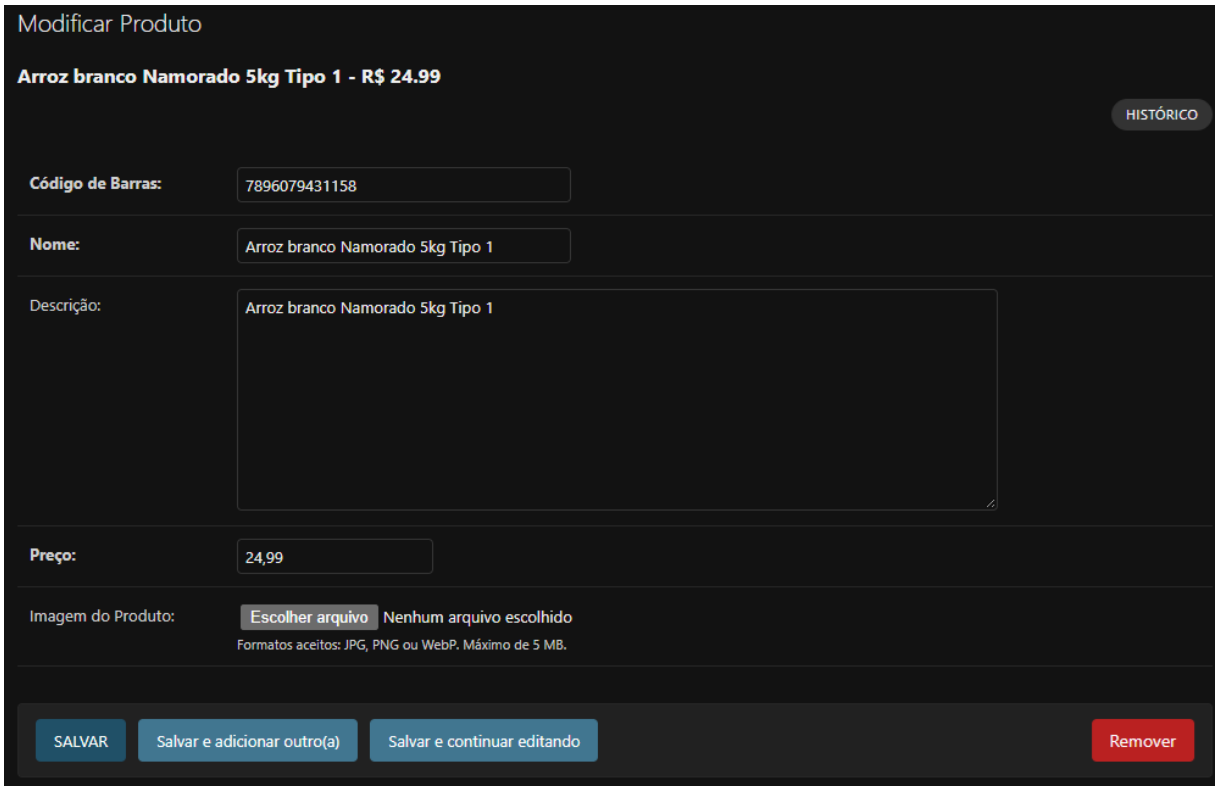
2.2 Funcionalidades do produto

A tela de leitura disponibiliza visualização da câmera, orientação para enquadramento do código de barras e um campo para pesquisa manual. O ML Kit reconhece os formatos EAN-13, EAN-8, UPC-A e UPC-E; após uma identificação válida, o aplicativo inicia a consulta sem exigir o acionamento de outro comando. A função Nova leitura limpa a consulta anterior e reativa a câmera.

A tela de resultado apresenta nome, descrição, código de barras, preço em reais e imagem quando disponível. Os estados Inicial, Carregando, Sucesso e Erro são controlados pelo ViewModel. São exibidas mensagens apropriadas em situações de código inválido, produto inexistente e indisponibilidade do servidor. A interface também permite consultar produtos sem fotografia cadastrada.

O painel administrativo (Figura 1) disponibiliza manutenção individual dos produtos e envio de imagens. Os códigos de barras são mantidos como texto para preservar zeros iniciais, enquanto o preço é armazenado como valor decimal no banco e transmitido como texto decimal no JSON. O endpoint público de consulta é GET /api/produtos/{codigo_barra}/, com retornos HTTP 200, 400 ou 404.

Figura 1 – Interface administrativa de cadastro e edição de produtos.



Fonte: captura de tela do sistema desenvolvido pelo autor (2026).

2.3 Ambiente operacional e tecnologias

O cliente foi desenvolvido para Android com Kotlin, Jetpack Compose e o padrão Model-View-ViewModel (MVVM). CameraX e ML Kit realizam captura e reconhecimento dos códigos; Retrofit envia as requisições; Moshi interpreta os dados JSON; e Coil carrega as imagens. A organização em camadas de interface, ViewModel e Repository segue princípios de separação de responsabilidades recomendados para aplicações Android (ANDROID DEVELOPERS, s.d.).

O servidor utiliza Python 3.14.2, Django 6.0.10, Django REST Framework 3.16.1 e MySQL 8.0.46. O Django ORM executa as consultas, e o Django Admin oferece manutenção autenticada. A API é documentada em OpenAPI 3.0.3, gerado por drf-spectacular, com interface Swagger UI. As imagens são armazenadas como arquivos WebP, separados da tabela de produtos.

A validação física foi realizada em um Motorola Moto G22, com Android 12, conectado por Wi-Fi ao servidor Windows. O aplicativo utiliza endereço HTTP configurado no BuildConfig para o ambiente de desenvolvimento. Em uma implantação externa, a segurança de transporte, a disponibilidade e a compatibilidade com outras versões do sistema operacional devem ser avaliadas separadamente.

3. METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE

Adotou-se uma abordagem incremental com organização visual do trabalho por Kanban, adequada ao desenvolvimento individual e à necessidade de priorizar entregas verificáveis. As tarefas foram organizadas no Trello em quatro colunas: Pendente, Em Processo, Em Pausa e Concluído. O trabalho em andamento foi limitado a duas tarefas simultâneas para favorecer o acompanhamento e reduzir mudanças paralelas (ATLASSIAN, s.d.).

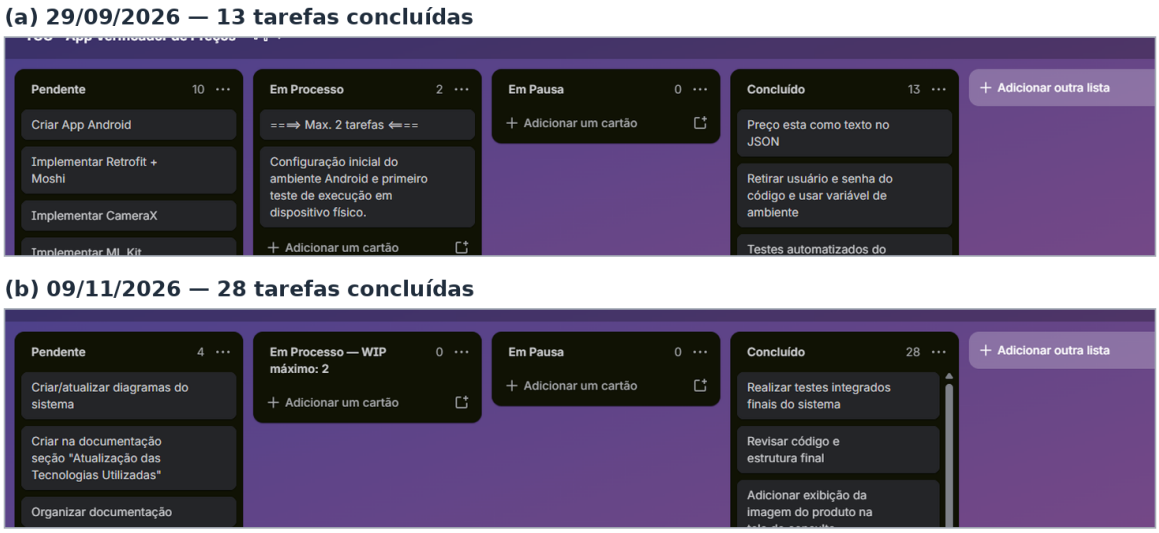
O desenvolvimento foi dividido em etapas progressivas: modelagem e configuração do backend, implementação da API REST, construção da interface Android, integração da câmera, gerenciamento de imagens e revisão de consistência. Cada etapa compreendeu alteração controlada, execução de testes e avaliação do comportamento antes da evolução seguinte. Durante o desenvolvimento, foram utilizadas ferramentas de inteligência artificial generativa, como ChatGPT da OpenAI, como apoio à análise técnica e revisão de código-fonte, organização da documentação, revisão textual e criação de imagens para melhor ilustrar diagramas e fluxos. As contribuições dessas ferramentas foram submetidas à avaliação do autor,

enquanto os resultados funcionais foram verificados por testes automatizados e integrados.

Os testes foram aplicados em diferentes níveis. O backend utilizou testes automatizados Django/DRF com banco MySQL separado e mídia temporária; no Android foram empregados testes unitários com repositório simulado; por fim, foram realizados testes integrados em dispositivo físico e rede local. Esse procedimento permitiu verificar os componentes individualmente e validar o percurso da consulta real.

A documentação técnica registrou as decisões e os resultados de cada etapa. Os registros históricos do Trello (Figura 2) mostram o avanço de 13 tarefas concluídas em 29 de setembro para 28 em 9 de novembro de 2026; a comparação evidencia o acompanhamento incremental, sem constituir, isoladamente, uma métrica da qualidade do software.

Figura 2 – Evolução das atividades do projeto no Trello entre setembro e novembro de 2026.



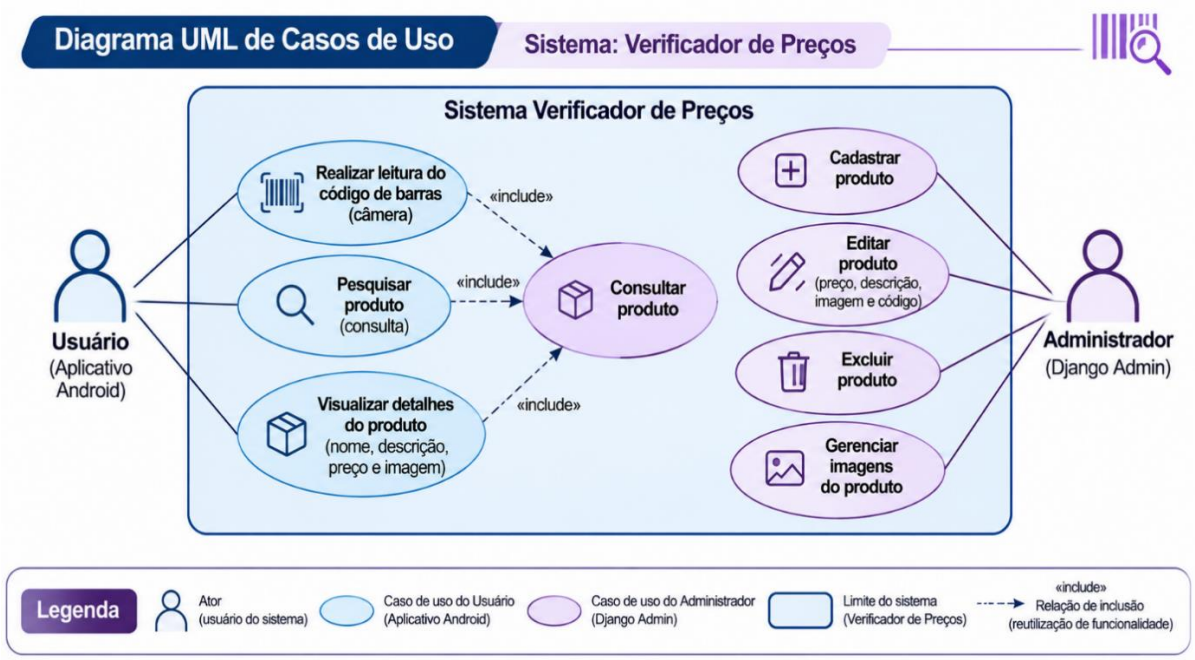
Fonte: registros do autor no Trello (2026).

4. DESENVOLVIMENTO

4.1 Modelagem e casos de uso

A modelagem concentrou-se na entidade Produto, com os campos id, codigo_barra, nome, descricao, preco e data_atualizacao. O código de barras é único e utilizado como chave de pesquisa da API. A imagem não é armazenada como coluna do MySQL: o arquivo WebP recebe o nome do código de barras e é localizado pelo backend ao compor a resposta. O diagrama de casos de uso (Figura 3) distingue o usuário que consulta produtos do administrador responsável pela manutenção dos cadastros.

Figura 3 – Diagrama UML de casos de uso do Verificador de Preços.



Fonte: gerada por ChatGPT em 9 nov. 2026.

4.2 Arquitetura e integração

A arquitetura separa o aplicativo Android, o backend Django e o armazenamento de dados. Na interface, o Jetpack Compose apresenta as telas de leitura e resultado; o ProdutoViewModel administra estados e cancelamentos; o ProdutoRepository concentra o acesso aos dados; e Retrofit/Moshi executam as requisições e a conversão das respostas. Essa divisão reduz o acoplamento entre interface e serviços de comunicação.

A API REST recebe o código de barras, valida seu formato, consulta o modelo Produto pelo Django ORM e devolve os dados em JSON. Quando há imagem, a resposta inclui uma URL; o Coil obtém o arquivo WebP por requisição própria. A Figura 5 representa

as responsabilidades e o armazenamento independente de registros relacionais e arquivos de imagem.

A interface ProdutoApi, apresentada na Figura 4, define a requisição HTTP GET utilizando Retrofit. A anotação @Path insere o código de barras na rota da API, enquanto a função suspensa consultarProduto() retorna uma resposta tipada como Response<ProdutoDto>, permitindo tratar os resultados e os códigos de status HTTP.

Figura 4 – Interface Retrofit para consulta de produtos à API REST.

```kotlin
// ProdutoApi.kt
package com.tcc.verificadorprecos.network

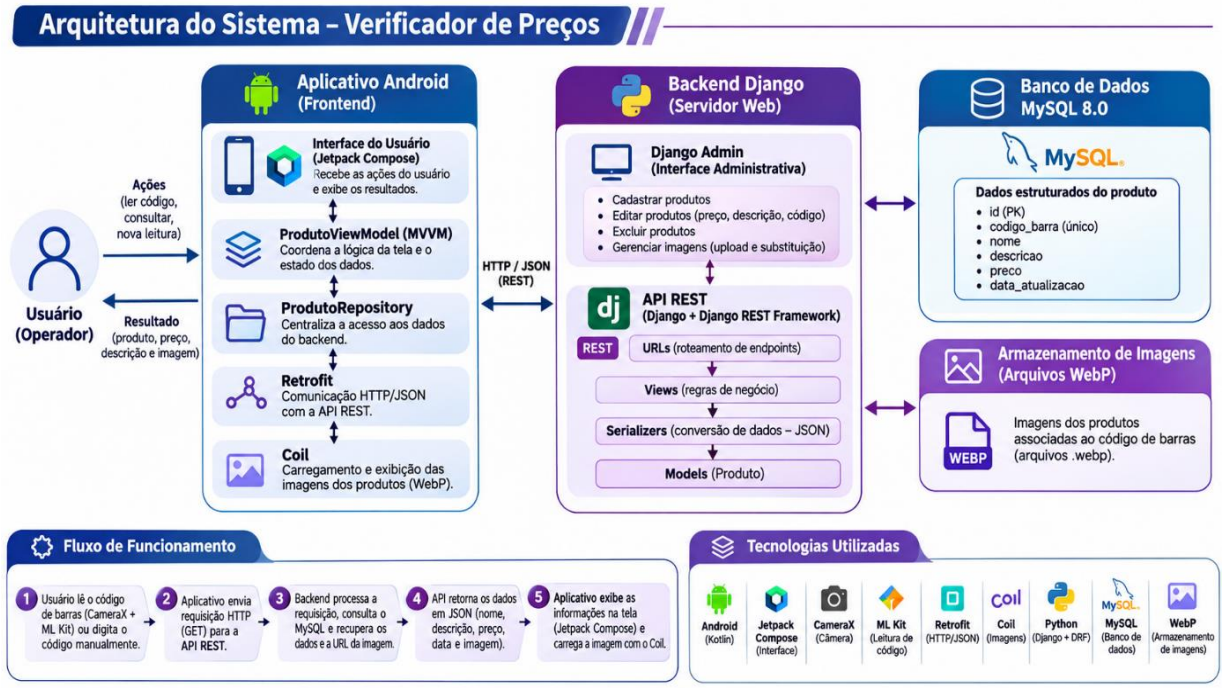
import com.tcc.verificadorprecos.data.ProdutoDto
import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Path

4 Usages
interface ProdutoApi {

    1 Usage
    @GET("api/produtos/{codigo_barra}/")
    suspend fun consultarProduto(
        @Path( value = "codigo_barra") codigoBarra: String
    ): Response<ProdutoDto>
}
```

Fonte: captura de tela do código-fonte do autor (2026).

Figura 5 – Arquitetura do sistema Verificador de Preços.



Fonte: gerada por ChatGPT em 9 nov. 2026.

4.3 Implementação e consistência dos dados

O backend oferece cadastro, edição e exclusão individual de produtos por meio do Django Admin. A API distingue sucesso (HTTP 200), código inválido (HTTP 400) e produto não encontrado (HTTP 404); a especificação OpenAPI descreve os campos, inclusive imagem como URL ou valor nulo. A configuração de conexão ao banco utiliza variáveis de ambiente, evitando inserir credenciais no código-fonte.

Para imagens, foi adotada validação prévia de arquivos JPEG, PNG ou WebP, com limites de tamanho e resolução, correção de orientação e conversão para WebP em memória. A gravação usa arquivo temporário e substituição por os.replace(). A publicação e a remoção são agendadas após confirmação da transação por transaction.on_commit(), de modo que um rollback do cadastro não substitua indevidamente a imagem anterior (DJANGO SOFTWARE FOUNDATION, s.d.).

Também foram incluídas verificações contra arquivos de imagem órfãos e proteções no processo de exclusão. Como MySQL e sistema de arquivos não compartilham uma transação única, falhas após o commit ainda podem exigir regularização manual; essa limitação é reconhecida no escopo do protótipo.

4.4 Aplicativo Android e experiência de consulta

O uso da câmera foi implementado com CameraX e reconhecimento local de códigos pelo ML Kit, evitando requisições repetidas do mesmo código durante uma sessão de leitura. A aplicação permite alternar para consulta digitada, apresenta mensagens de carregamento e erro e reinicia o fluxo pela função Nova leitura. O controle do ciclo de vida libera recursos da câmera ao sair da tela de leitura, enquanto o ViewModel cancela consultas anteriores para impedir resultados desatualizados (GOOGLE, s.d.).

As telas foram organizadas com contraste entre informações gerais e preço, exibição opcional de fotografia e rolagem vertical. As capturas atuais da Figura 6 mostram a interface de leitura por câmera e a consulta bem-sucedida de um produto real, com imagem, código de barras, descrição e preço. O estado de entrada digitada passou a utilizar rememberSaveable para preservar o valor em recriações da Activity, incluindo zeros iniciais.

Figura 6 – Telas do aplicativo Android: leitura e resultado da consulta.



Fonte: capturas de tela do aplicativo desenvolvido pelo autor (2026).

4.5 Testes e resultados obtidos

A etapa de validação final reuniu 115 testes automatizados do backend, que cobriram API, validações, imagens e transações, e 25 testes unitários Android, com verificações do ViewModel e do analisador de códigos. As suítes foram executadas com sucesso e o Android Lint não registrou erros. O OpenAPI 3.0.3 também foi validado sem avisos.

Tabela 1 – Resultados dos testes automatizados e integrados do sistema.

Nível de validação	Executados	Aprovados
Backend Django / DRF	115	115
Aplicativo Android (unitários)	25	25
Integração em dispositivo físico	13	13
Total de cenários informados	153	153

Fonte: resultados dos testes realizados pelo autor (2026).

Nos 13 cenários integrados, foram validados conexão Wi-Fi, consulta manual e por câmera, produtos com e sem imagem, atualização de preços, alteração de imagens e códigos de barras, produto inexistente, código inválido, Nova leitura, falhas de comunicação, exclusão e limpeza de dados temporários. Todos os testes apresentaram os resultados esperados no Motorola Moto G22 (Android 12), preservando os registros originais e removendo os produtos temporários.

A interface diferenciou códigos inválidos de produtos não cadastrados, conforme a Figura 7. No contrato da API, essas situações correspondem aos códigos HTTP 400

e 404, respectivamente, embora entradas inválidas possam ser bloqueadas pela interface antes do envio da requisição.

Figura 7 – Tratamento de erros: (a) produto não encontrado e (b) código de barras inválido.



Fonte: capturas de tela do aplicativo desenvolvido pelo autor (2026).

o conjunto de evidências confirma o funcionamento do fluxo proposto no ambiente ensaiado. Os resultados são de testes funcionais e de regressão; não constituem estudo estatístico de desempenho, ensaio de usabilidade com participantes nem homologação para uso em produção.

Repositório público do código-fonte:

https://github.com/marlomlucas/tcc

5. CONSIDERAÇÕES FINAIS

O trabalho atingiu seu objetivo de desenvolver e validar um protótipo Android para consulta de informações de produtos por código de barras. A solução integra reconhecimento pela câmera, interface móvel, API REST e banco relacional, permitindo identificar mercadorias e apresentar preços e imagens em rede local, com consulta manual alternativa.

A execução incremental e a separação entre interface, lógica de estados, comunicação e persistência contribuíram para a organização do software. No backend, a documentação OpenAPI e as validações de transação e armazenamento de imagens ampliaram a previsibilidade das operações administrativas. Os 140 testes automatizados e os 13 cenários integrados aprovados demonstraram a estabilidade funcional do conjunto nos casos examinados.

A avaliação está delimitada ao Motorola Moto G22 com Android 12 e ao servidor executado em ambiente de desenvolvimento por Wi-Fi e HTTP. Não foram realizados testes abrangentes de desempenho sob carga, segurança de implantação pública, acessibilidade com usuários ou compatibilidade em diferentes versões do Android. A solução também depende de cadastros atualizados pelo administrador; não consulta automaticamente sistemas comerciais de terceiros.

Como evolução futura, propõem-se a disponibilização do backend em ambiente seguro com HTTPS, testes em outras versões e dispositivos Android, melhoria da adaptação a telas com barras do sistema e avaliação do suporte a novos formatos, como QR Code, caso exista necessidade funcional. Assim, o protótipo constitui uma demonstração aplicada de conceitos de Engenharia de Software e uma base para futuras ampliações.

REFERÊNCIAS

ANDROID DEVELOPERS. Recomendações para a arquitetura do Android. Disponível em: https://developer.android.com/topic/architecture/recommendations?hl=pt-br. Acesso em: 9 nov. 2026.

ATLASSIAN. O que é o Kanban na gestão de projetos? Disponível em: https://www.atlassian.com/br/agile/kanban. Acesso em: 9 nov. 2026.

BRASIL. Lei nº 10.962, de 11 de outubro de 2004. Dispõe sobre a oferta e as formas de afixação de preços de produtos e serviços para o consumidor. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2004/lei/l10.962.htm. Acesso em: 9 nov. 2026.

DJANGO SOFTWARE FOUNDATION. Database transactions: performing actions after commit. Django 6.0 documentation. Disponível em: https://docs.djangoproject.com/en/6.0/topics/db/transactions/. Acesso em: 9 nov. 2026.

GOOGLE. Scan barcodes with ML Kit on Android. Google for Developers. Disponível em: https://developers.google.com/ml-kit/vision/barcode-scanning/android. Acesso em: 9 nov. 2026.

OPENAI. *ChatGPT: apoio na organização de ideias e revisão da documentação do projeto Verificador de Preços*. Inteligência artificial generativa. Disponível em: https://chatgpt.com/. Acesso em: 12 nov. 2026.